Documento de Diseño Preliminar: Comparación de Estructuras Candidatas

Para resolver el requerimiento de gestionar títulos con distinto número de ejemplares (evitando celdas vacías y soportando ordenamiento manual y búsqueda binaria), se analizan tres opciones estructurales en C#:

1. Arreglo Escalonado (int[][] **o** string[][]**)**

Un arreglo de arreglos donde cada fila representa un título y las columnas contienen los ejemplares asociados o sus datos.

- **Pros:**
    - **Cero desperdicio de memoria:** Al ser una estructura escalonada (jagged array), cada fila se inicializa únicamente con la cantidad exacta de ejemplares de ese título, cumpliendo estrictamente con la restricción de no dejar celdas vacías.
    - **Acceso directo por índice:** Permite un rendimiento de acceso $O(1)$ una vez localizada la fila del libro.
- **Contras:**
    - **Dificultad de mantenimiento y baja cohesión:** Al manipular tipos primitivos en arreglos primarios y secundarios, la información del título (ISBN, préstamos, nombre) debe separarse en otros arreglos o codificarse

mediante posiciones, lo que dificulta la legibilidad y el ordenamiento manual por inserción.

- **Estructura rígida:** Redimensionar o agregar ejemplares requiere reasignar memoria manualmente.

2. Lista de Arreglos (List<int[]>**)**

Una lista dinámica donde cada elemento representa un título mediante un arreglo numérico de tamaño variable con sus ejemplares.

- **Pros:**
  - **Flexibilidad de tamaño a nivel de catálogo:** Permite agregar o eliminar títulos del catálogo fácilmente gracias al uso de List<T>.
  - **Ajuste exacto por título:** Cada sub-arreglo int[] almacena únicamente los ejemplares requeridos sin dejar celdas vacías.
- **Contras:**
  - **Falta de expresividad del dominio:** Al usar arreglos int[], se pierde claridad sobre qué representa cada valor (ejemplo: separar préstamos históricos, ISBN y disponibilidad requiere convenciones de índices explícitas como posicion[0], posicion[1]).

- **Mayor complejidad en ordenamientos dinámicos:** Ordenar una lista de arreglos primitivos manualmente requiere manipular referencias y sub-índices de manera menos intuitiva.

3. Lista de Objetos (List<Título>**)**

Una lista dinámica de objetos pertenecientes a una clase personalizada Título. Dentro de la clase Título, los ejemplares se almacenan en una colección interna de tamaño ajustado (por ejemplo, un arreglo int[] o Ejemplar[] instanciado a la medida exacta).

- **Pros:**

  - **Alta cohesión y diseño orientado a objetos:** Encapsula en una sola entidad el ISBN, el nombre del título, el contador de préstamos históricos y su listado interno de ejemplares.

  - **Facilidad para implementar el ordenamiento e intercambio:** Al aplicar el ordenamiento por inserción manual, mover un elemento en la lista implica intercambiar únicamente la referencia del objeto Título, facilitando el control del criterio principal (préstamos) y del criterio secundario (orden alfabético para estabilidad).

  - **Cumplimiento total de restricciones:** Permite que la lista interna de ejemplares de cada objeto tenga exactamente la longitud de sus copias (1 a 5 copias), sin celdas vacías.

- **Contras:**

    - Ligeramente mayor overhead de memoria por la sobrecarga propia del objeto en el Heap de C# (insignificante para el volumen de datos de la biblioteca).

Tabla Comparativa Resumen

Criterio	**Arreglo Escalonado (int[][])**	**List<int[]>**	**List<Título>**
Ausencia de celdas vacías	Cumple (tamaño dinámico por fila)	Cumple (arreglos a la medida)	Cumple (atributo interno ajustado)
Modelado y claridad del código	Muy bajo (datos dispersos)	Bajo (índices numéricos abstractos)	Excelente (POO, encapsulamiento)
Facilidad para Ordenamiento por Inserción	Compleja	Media	Alta (intercambio directo de referencias)
Búsqueda por ISBN / Algoritmos	Compleja	Compleja	Alta (acceso directo a atributos)

Selección de la Estructura

Estructura Elegida: List<Título>

Justificación: List<Título> es la opción óptima porque permite encapsular todos los atributos de un libro (ISBN, préstamos, ejemplares de tamaño dinámico) en un solo modelo de dominio. Cumple rigurosamente con la restricción de **no dejar celdas vacías** instanciando la estructura de ejemplares de cada objeto al tamaño exacto de copias. Además, simplifica significativamente la implementación manual del **ordenamiento por inserción** y de la **búsqueda binaria por ISBN**, garantizando un código legible, mantenible y modular para las siguientes fases del proyecto.